

Comenzado el	miércoles, 15 de enero de 2025, 01:35
Estado	Finalizado
Finalizado en	miércoles, 15 de enero de 2025, 01:53
Tiempo empleado	17 minutos 8 segundos
Puntos	21,67/25,00
Calificación	8,67 de 10,00 (86,67%)

Pregunta 1

Correcta

Se puntúa 1,00 sobre 1,00

En el algoritmo de Dijkstra con un grafo con V vértices y A aristas, ¿cuántas veces se invocan como mínimo las operaciones insertar y borrar el mínimo de la cola por cada vértice?

Seleccione una:

- ☐ a. V
- ☒ b. 0 ✓
- ☐ c. 2
- ☐ d. 1

Los vértices no alcanzables desde el origen no se insertan nunca en la cola de prioridad.

La respuesta correcta es: 0

Pregunta 2

Correcta

Se puntúa 1,00 sobre 1,00

Si el TAD de conjuntos disjuntos se implementa con la versión de *búsqueda rápida*, ¿cuál es el coste de la operación **unir** en el caso peor si el conjunto tiene N elementos?

Seleccione una:

- ☐ a. $O(1)$
- ☐ b. $O(N^2)$
- ☐ c. $O(\log N)$
- ☒ d. $O(N)$ ✓

Al unir dos conjuntos hay que buscar en todo el vector para mover los elementos de un conjunto al otro.

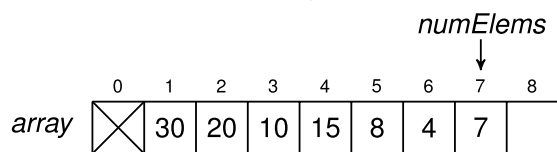
La respuesta correcta es: $O(N)$

Pregunta 3

Correcta

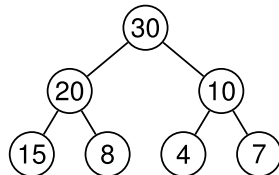
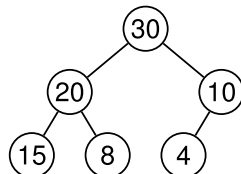
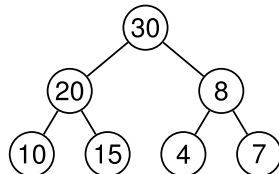
Se puntúa 1,00 sobre 1,00

¿Qué montículo de máximos representa este vector?



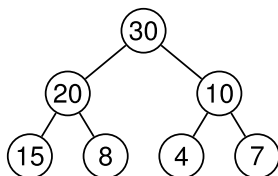
Seleccione una:

☐ a. No representa un montículo de máximos correcto.

☒ b. ✓

☐ c.

☐ d.


La raíz está en la posición 1, y el nodo de la posición i tiene su hijo izquierdo en la posición $2i$ y su hijo derecho en la posición $2i + 1$, si estos números no exceden *numElems*. Todo nodo cumple que es mayor (o igual) que sus hijos, si estos existen.

La respuesta correcta es:

**Pregunta 4**

Correcta

Se puntúa 1,00 sobre 1,00

Queremos planificar un conjunto de tareas para ejecutarlas en un único procesador de forma que se minimice su tiempo medio de estancia en el sistema (espera más ejecución). Si estas tareas son 7 y tienen tiempos de ejecución 26, 15, 29, 18, 25, 13 y 19, ¿en qué orden ejecutarías esas tareas para minimizar el tiempo medio? Escribe los tiempos de las tareas en el orden en el que se ejecutarían.

 Respuesta: ✓

La planificación adecuada es la secuencia de tareas ordenada por tiempos de menor a mayor (es decir, 13, 15, 18, 19, 25, 26 y por último 29). El tiempo medio es entonces $503/7$.

La respuesta correcta es: 13 15 18 19 25 26 29

Pregunta 5

Correcta

Se puntúa 1,00 sobre 1,00

El algoritmo voraz para resolver el problema de la mochila con n objetos que se pueden fraccionar y peso límite también n tiene un coste en:

Seleccione una:

- ☐ a. $O(n^2)$
- ☒ b. $O(n \log n)$ ✓
- ☐ c. $O(n)$
- ☐ d. $O(n^2 \log n)$

El coste del algoritmo voraz que resuelve el problema de la mochila real es independiente del peso límite. El coste está dominado por la fase de ordenar los objetos de mayor a menor valor por unidad de peso, que está en $O(n \log n)$ siendo n el número de objetos. La segunda fase, de selección de los objetos, los recorre todos en el caso peor y tiene coste en $O(n)$.

La respuesta correcta es: $O(n \log n)$

Pregunta 6

Correcta

Se puntúa 1,00 sobre 1,00

El *traspuesto* (o *inverso*) de un grafo $G = (V, A)$ es otro grafo $G^T = (V, A^T)$ donde $A^T = \{(v, u) \mid (u, v) \in A\}$.

¿Qué complejidad tendría un algoritmo para calcular el grafo traspuesto de un grafo dirigido representado mediante *listas de adyacentes* si el grafo tiene V vértices y A aristas?

Seleccione una:

- ☐ a. $O(V^2)$
- ☒ b. $O(V + A)$ ✓
- ☐ c. $O(V * A)$
- ☐ d. $O(V)$

Para calcular el grafo traspuesto podemos recorrer las listas de adyacentes de G y si encontramos el vértice v en la lista de adyacentes a u , añadir u a los adyacentes a v en el grafo G^T . Como hay V listas y el número total de elementos en todas ellas es A , el coste total está en $O(V + A)$.

La respuesta correcta es: $O(V + A)$

Pregunta 7

Correcta

Se puntúa 1,00 sobre 1,00

Sea el problema de la mochila entera con los objetos siguientes, donde la primera componente es el valor y la segunda el peso: $(40, 2)$, $(30, 5)$, $(50, 10)$, $(10, 5)$. Si la capacidad de la mochila es 16, al final de la ejecución del algoritmo de ramificación y poda que utiliza el algoritmo voraz para calcular la prioridad de los nodos:

Seleccione una:

- ☒ a. quedan tres nodos en la cola de prioridad. ✓
- ☐ b. la cola de prioridad queda vacía.
- ☐ c. queda un nodo en la cola de prioridad.
- ☐ d. quedan dos nodos en la cola de prioridad.

Representando en cada nodo el valor, peso y estimación de la solución parcial por una tripla $(valor, peso, estimación)$

1- Inicialmente en la cola solo está el nodo raíz $(0, 0, 115)$.

2- Sale ese nodo y entran $nodo_1 = (40, 2, 115)$ y $nodo_2 = (0, 0, 82)$.

3- Sale $nodo_1$ y entran $nodo_3 = (70, 7, 115)$ y $nodo_4 = (40, 2, 98)$.

4- Sale $nodo_3$ y entra $nodo_5 = (70, 7, 80)$.

5- Sale $nodo_4$ y entran $nodo_6 = (90, 12, 98)$ y $nodo_7 = (40, 2, 50)$.

6- Sale $nodo_6$ y su hijo $nodo_8 = (90, 12, 90)$ es la primera solución.

7- Puesto que el valor de esta solución es mayor o igual que las prioridades de los nodos que están en la cola, el algoritmo termina, quedando en la cola tres nodos: $nodo_2$, $nodo_5$ y $nodo_7$.

Por tanto la respuesta correcta es que quedan 3 nodos en la cola de prioridad.

La respuesta correcta es: quedan tres nodos en la cola de prioridad.

Pregunta 8

Correcta

Se puntúa 1,00 sobre 1,00

¿Cuál es el coste de añadir una arista en un grafo no dirigido con V vértices y A aristas, representado mediante *listas de adyacentes*?

Seleccione una:

- ☐ a. $O(V)$
- ☐ b. $O(V * A)$
- ☐ c. $O(V + A)$
- ☒ d. $O(1)$ ✓

Para añadir la arista $u - v$ basta con añadir v a la lista de adyacentes de u y u a la lista de adyacentes de v , lo cual se puede hacer en tiempo constante.

La respuesta correcta es: $O(1)$

Pregunta 9

Correcta

Se puntúa 1,00 sobre 1,00

Supongamos un sistema monetario con cantidad ilimitada de monedas de valores {2, 3, 5, 7}. Se desea pagar la cantidad 9 con el menor número de monedas. Los índices de la última fila de la matriz rellenada por el algoritmo que contienen un infinito son:

Seleccione una:

- ☐ a. 2, 6, 7
- ☐ b. 4, 7
- ☒ c. 1 ✓
- ☐ d. 1, 2

Las filas se rellenan de arriba abajo y de izquierda a derecha obteniendo la siguiente tabla:

	0	1	2	3	4	5	6	7	8	9
0	0	∞	∞	∞	∞	∞	∞	∞	∞	∞
1	0	∞	1	∞	2	∞	3	∞	4	∞
2	0	∞	1	1	2	2	2	3	3	3
3	0	∞	1	1	2	1	2	2	2	3
4	0	∞	1	1	2	1	2	1	2	2

Por tanto la solución es: 1

La respuesta correcta es: 1

Pregunta 10

Correcta

Se puntúa 1,00 sobre 1,00

Si el grafo está representado mediante una *matriz de adyacencia* y el método $ady(v)$ devuelve una lista con los vértices adyacentes a v , ¿cuál es la complejidad del siguiente algoritmo que calcula el número de aristas del grafo si este tiene V vértices y A aristas?

```
Grafo grafo(V);
int aristas = 0;
for (int v = 0; v < V; ++v)
    aristas += grafo.ady(v).size();
cout << aristas/2 << '\n';
```

Seleccione una:

- ☐ a. $O(V + A)$
- ☒ b. $O(V^2)$ ✓
- ☐ c. $O(V)$
- ☐ d. $O(V * A)$

La operación para calcular los adyacentes a un vértice tiene coste en $O(V)$ (aunque después de la lista solamente queramos saber su longitud). Y esta operación se repite para cada vértice. El coste total está en $O(V^2)$.

La respuesta correcta es: $O(V^2)$

Pregunta 11

Correcta

Se puntúa 1,00 sobre 1,00

El problema de la multiplicación encadenada de n matrices de forma óptima puede resolverse con una complejidad en tiempo en:

Seleccione una:

- ☐ a. $O(m^2)$ siendo m el número de filas y columnas de las n matrices
- ☒ b. $O(n^3)$ ✓
- ☐ c. $O(n^2)$
- ☐ d. $O(m^3)$ siendo m el número de filas y columnas de las n matrices

El algoritmo calcula para cada pareja de índices i, j tales que $i \leq j$ el número mínimo de multiplicaciones básicas para multiplicar las matrices de la i a la j . El cálculo de cada valor es lineal en n porque requiere considerar las distintas formas de colocar los paréntesis externos para realizar la última multiplicación. Puesto que la cantidad de parejas a calcular es del $O(n^2)$, el coste está en $O(n^3)$.

La respuesta correcta es: $O(n^3)$

Pregunta 12

Correcta

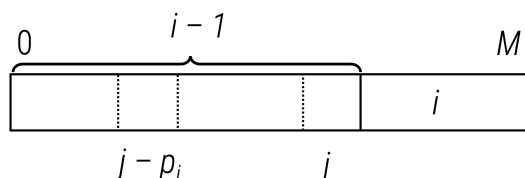
Se puntúa 1,00 sobre 1,00

Si queremos reducir el coste en espacio del algoritmo de programación dinámica que resuelve la versión entera del problema de la mochila (cuando solamente estamos interesados en el valor óptimo y no en los objetos que hay que meter en la mochila) utilizando un vector en lugar de una matriz, debemos rellenar el vector:

Seleccione una:

- ☐ a. tanto de derecha a izquierda como de izquierda a derecha.
- ☐ b. de izquierda a derecha.
- ☒ c. de derecha a izquierda. ✓
- ☐ d. no se puede reducir el coste en espacio.

En cada iteración del algoritmo en que consideramos el objeto i -ésimo rellenamos un vector de $M + 1$ posiciones, siendo M el peso límite de la mochila. Al empezar la iteración i -ésima, el vector contiene los valores correspondientes al objeto $i - 1$. Puesto que para calcular los valores correspondientes al objeto i y cada peso límite j necesitamos dos valores correspondientes al objeto $i - 1$ en columnas j y $j - p_i$ (siendo p_i el peso del objeto i), se ha de rellenar el vector de *derecha a izquierda* para mantener dichos valores disponibles hasta realizar el cálculo, tal como se muestra en el siguiente dibujo:



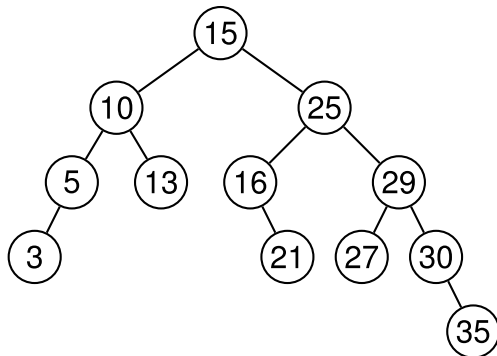
La respuesta correcta es: de derecha a izquierda.

Pregunta 13

Sin contestar

Se puntúa como 0 sobre 1,00

Si en este árbol AVL eliminamos el valor 16, ¿qué tipo de rotaciones se producen?



Seleccione una o más de una:

- ☐ a. Rotación doble derecha-izquierda.
- ☐ b. Ninguna, tras eliminar está equilibrado.
- ☐ c. Rotación simple a la izquierda.
- ☐ d. Rotación doble izquierda-derecha.

Al eliminar el valor 16 este es sustituido por el valor 21, lo que provoca que el nodo con valor 25 pierda la condición de equilibrio, y haga falta una rotación simple a la izquierda para restablecerla. Esto hace que el árbol pase a estar equilibrado.

La respuesta correcta es: Rotación simple a la izquierda.

Pregunta 14

Correcta

Se puntúa 1,00 sobre 1,00

A la hora de convertir un vector en un montículo como primera fase para ordenarlo de menor a mayor por el método heapsort, ¿cuál es el caso peor (el caso que provoca que haya que hacer más trabajo)?

Seleccione una:

- ☐ a. No se puede saber a priori.
- ☐ b. Todos los casos provocan la misma cantidad de trabajo.
- ☒ c. Cuando el vector inicial está ordenado de menor a mayor. ✓
- ☐ d. Cuando el vector inicial está ordenado de mayor a menor.

Para ordenar de menor a mayor el vector tiene que convertirse en un montículo de máximos. El caso peor se da cuando el vector está inicialmente ordenado de menor a mayor porque cada elemento (en la primera mitad del vector) es menor que todos los siguientes y tiene que ser hundido hasta una hoja.

La respuesta correcta es: Cuando el vector inicial está ordenado de menor a mayor.

Pregunta 15

Correcta

Se puntúa 1,00 sobre 1,00

Sea G un grafo dirigido con todos los valores de las aristas iguales. Podemos calcular de manera más eficiente el camino más corto entre un origen y un destino utilizando

Seleccione una:

- ☐ a. Búsqueda en profundidad (DFS).
- ☐ b. No se puede utilizar ni BFS ni Dijkstra.
- ☒ c. Búsqueda en anchura (BFS). ✓
- ☐ d. El algoritmo de Dijkstra.

Si los costes de las aristas son todos iguales, el camino más corto es el que menos aristas utiliza. El coste en tiempo del algoritmo de Dijkstra está en $O(A * \log V)$, mientras que el algoritmo BFS tiene un coste en tiempo en $O(A + V)$.

Por lo tanto, la forma más eficiente de encontrar estos caminos es utilizando búsqueda en anchura (BFS).

La respuesta correcta es: Búsqueda en anchura (BFS).

Pregunta 16

Incorrecta

Se puntúa -0,33 sobre 1,00

Tenemos una pared en la que queremos colocar el mayor número de rollos con papel pintado sin que se solapen. Los tramos de papel pintado los colocaremos en vertical.

Considerando que la longitud de cada rollo de papel es la misma (y la necesaria para cubrir completamente de arriba a abajo la altura de la pared), pero su anchura no. ¿Cuál de las siguientes estrategias voraces es correcta para resolver el problema anterior?

Seleccione una:

- ☐ a. Ordenamos los rollos por orden creciente de anchura y cogemos el primero libre que quepa
- ☒ b. Ordenamos los rollos por orden decreciente de anchura y elegimos el primero libre que quepa ✗ Falso. Incorrecta. Supongamos una pared de 70cm y tres rollos de 70cm, 50cm y 20cm. La estrategia propuesta sólo coge el primero y la óptima los dos últimos.
- ☐ c. Ordenamos los rollos por orden creciente en su longitud y elegimos el primero libre que quepa
- ☐ d. Ninguna de las anteriores.

- a. Cierto. Intuitivamente estamos cogiendo aquellos rollos que más hueco libre nos dejan para después. Se puede demostrar mediante el método de reducción de diferencias que dicha estrategia es óptima para resolver el problema.
- b. Falso. Incorrecta. Supongamos una pared de 70cm y tres rollos de 70cm, 50cm y 20cm. La estrategia propuesta sólo coge el primero y la óptima los dos últimos.
- c. Falso. Incorrecta. La longitud de los rollos es la misma por lo que esa ordenación no nos ayuda. Por ejemplo, en una pared de 70cm y tres rollos de 70cm, 50cm y 20cm. La estrategia propuesta cogería sólo el primero y la óptima los dos últimos.
- d. Falso. La respuesta correcta es: Ordenamos los rollos por orden creciente de anchura y cogemos el primero libre que quepa

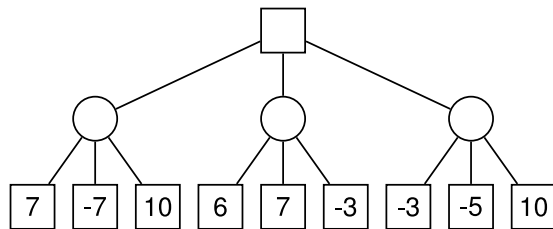
La respuesta correcta es: Ordenamos los rollos por orden creciente de anchura y cogemos el primero libre que quepa

Pregunta 17

Correcta

Se puntúa 1,00 sobre 1,00

¿Cuál es el valor de la raíz de este árbol de juego si aplicamos el algoritmo minimax?



Seleccione una:

- ☐ a. 0
- ☐ b. 10
- ☐ c. -5
- ☒ d. -3 ✓

El algoritmo minimax calcula desde las hojas hacia la raíz alternando mínimos y máximos. Los valores de los nodos con forma de círculo en el dibujo se calculan haciendo el mínimo de los valores de sus hijos. De izquierda a derecha esos valores son -7, -3 y -5. A continuación se calcula el valor de la raíz, que es el máximo de estos tres valores, por lo que la respuesta correcta es -3.

La respuesta correcta es: -3

Pregunta 18

Correcta

Se puntúa 1,00 sobre 1,00

Dado un grafo dirigido implementado con *listas de adyacencia*, ¿cuál sería el coste de un algoritmo eficiente que busque un vértice con grado de entrada máxima, es decir, con grado de entrada $V - 1$?

Seleccione una:

- ☐ a. $O(1)$
- ☒ b. $O(A + V)$ ✓ Cierto. Para encontrar un nodo con grado de entrada $V - 1$, tenemos que recorreremos todas las listas de adyacencia de cada vértice para ver si algún vértice aparece $V - 1$ veces. Por lo tanto el coste es $O(A + V)$
- ☐ c. $O(A)$
- ☐ d. Ninguna de las anteriores.

- a. Falso. Incorrecta. Es $O(A + V)$.
- b. Cierto. Para encontrar un nodo con grado de entrada $V - 1$, tenemos que recorreremos todas las listas de adyacencia de cada vértice para ver si algún vértice aparece $V - 1$ veces. Por lo tanto el coste es $O(A + V)$
- c. Falso. Incorrecta. Es $O(A + V)$.
- d. Falso. La respuesta correcta es: $O(A + V)$

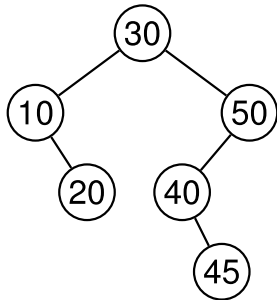
La respuesta correcta es: $O(A + V)$

Pregunta 19

Correcta

Se puntúa 1,00 sobre 1,00

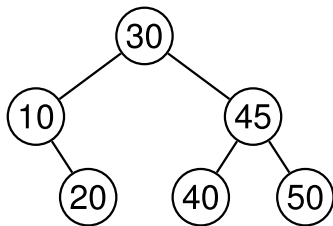
Tras insertar el valor 45, ¿qué tipo de rotación necesita este árbol?



Seleccione una:

- ☒ a. Rotación doble izquierda-derecha. ✓
- ☐ b. Rotación doble derecha-izquierda.
- ☐ c. Rotación simple a la izquierda.
- ☐ d. Rotación simple a la derecha.

El nodo con el valor 50 es el nodo α (el primero que no cumple la condición de equilibrio si vamos desde el nuevo nodo insertado hasta la raíz), y para equilibrarlo hace falta una rotación doble izquierda-derecha dando lugar al siguiente árbol



La respuesta correcta es: Rotación doble izquierda-derecha.

Pregunta 20

Correcta

Se puntúa 1,00 sobre 1,00

Hemos visto en *IndexPQ* una implementación de *colas con prioridad variable* que hace uso de un vector de pares $\langle \text{elemento}, \text{prioridad} \rangle$ y un vector de posiciones de esos elementos.

Imagina que modificamos esta implementación eliminando el vector de posiciones. ¿Cuál de estas afirmaciones es correcta?

Seleccione una:

- ☒ a. Modificar la prioridad (update) pasa a tener coste lineal.  Cierto. El montículo está ordenado según prioridades, no elementos, eso quiere decir que para modificar la prioridad de un elemento, antes, hay que encontrarlo. Como el elemento puede estar en cualquier posición hay que recorrer todos los elementos.
- ☐ b. Flotar pasa a tener coste lineal.
- ☐ c. Hundir pasa a tener coste lineal.
- ☐ d. Ninguna de las anteriores.

- a. Cierto. El montículo está ordenado según prioridades, no elementos, eso quiere decir que para modificar la prioridad de un elemento, antes, hay que encontrarlo. Como el elemento puede estar en cualquier posición hay que recorrer todos los elementos.
- b. Falso. Incorrecta. Sigue siendo logarítmica, flotamos un elemento ya encontrado.
- c. Falso. Incorrecta. Sigue siendo logarítmica, hundimos un elemento ya encontrado.
- d. Falso. La respuesta correcta es: Modificar la prioridad (update) pasa a tener coste lineal.

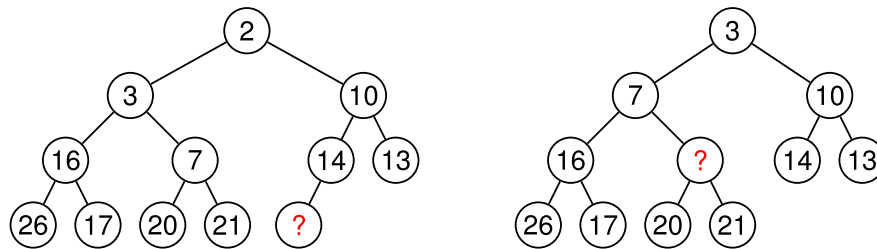
La respuesta correcta es: Modificar la prioridad (update) pasa a tener coste lineal.

Pregunta 21

Correcta

Se puntúa 1,00 sobre 1,00

La eliminación del mínimo en el siguiente montículo de mínimos de la izquierda da como resultado el montículo de la derecha:



¿Cuál podría ser el valor oculto (marcado con una interrogación) si sabemos que todos los valores son distintos?

Seleccione una:

- ☐ a. Podría ser cualquier entero (no tenemos suficiente información) distinto de los que ya están.
- ☐ b. Cualquier valor entre 8 y 19 (inclusive) que no esté ya.
- ☒ c. 15, 18 o 19. ✓ Cierto.
- ☐ d. Cualquier valor mayor que 14 que no esté ya.

Por ser hijo del 14 antes de la eliminación sabemos que tiene que ser mayor que 14. Por ser padre del 20, tiene que ser menor que 20. Como el 16 y el 17 ya están, solamente puede ser 15, 18 o 19.

- Falso. No puede ser el 1, por ejemplo, porque el 1 no puede ser hijo del 14.
- Falso. No puede ser el 9, por ejemplo, porque el 9 no puede ser hijo del 14.
- Cierto.
- Falso. No puede ser el 25, por ejemplo, porque el 25 no puede ser padre del 20 en un montículo de mínimos

La respuesta correcta es: 15, 18 o 19.

Pregunta 22

Correcta

Se puntúa 1,00 sobre 1,00

En el algoritmo de programación dinámica que resuelve el problema del producto encadenado de matrices, podemos rellenar la matriz:

Seleccione una:

- ☐ a. por diagonales comenzando en la diagonal del extremo superior derecho.
- ☒ b. por diagonales comenzando en la diagonal encima de la principal. ✓
- ☐ c. de arriba abajo y de derecha a izquierda.
- ☐ d. de arriba abajo y de izquierda a derecha.

Puesto que cada posición (i, j) depende de las posiciones (i, k) y $(k + 1, j)$ tales que $i \leq k \leq j - 1$, las cuales están respectivamente en su misma fila a la izquierda hasta la diagonal principal, y en la misma columna pero en filas inferiores hasta la diagonal principal, una posible forma de rellenar la matriz es por diagonales comenzando en la diagonal encima de la principal. El resto de opciones no garantiza que estén rellenas las posiciones necesarias. Sí podría rellenarse, sin embargo, de abajo arriba y de izquierda a derecha.

La respuesta correcta es: por diagonales comenzando en la diagonal encima de la principal.

Pregunta 23

Correcta

Se puntúa 1,00 sobre 1,00

¿Cuál de los siguientes juegos se pueden abordar con el algoritmo minimax visto en clase?

Seleccione una o más de una:

- ☒ a. Damas ✓ Este juego cumple todas las condiciones.
- ☐ b. La oca
- ☐ c. Piedra, papel o tijera
- ☒ d. 3 en raya ✓ Este juego cumple todas las condiciones.

El algoritmo minimax se aplica a juegos con información perfecta (en los que se conoce el estado del juego) donde el jugador y el adversario puede tomar decisiones. Eso excluye juegos de azar puro y solitarios.

- a. Este juego cumple todas las condiciones.
- b. Es un juego de azar puro.
- c. No es un juego por turnos (las acciones de los jugadores son simultáneas).
- d. Este juego cumple todas las condiciones.

Las respuestas correctas son: Damas, 3 en raya

Pregunta 24

Correcta

Se puntúa 1,00 sobre 1,00

Utilizando las estructuras de partición podemos detectar si un grafo es acíclico. Si están implementadas con *unión rápida por tamaño y compresión de caminos*, ¿cuál sería la complejidad de dicho algoritmo si el grafo tiene V vértices y A aristas?

Seleccione una:

- ☐ a. $O(VA)$
- ☐ b. $O(V \log V)$
- ☐ c. $O(V + A \log V)$
- ☒ d. $O(V + A \lg^* V)$ ✓

El algoritmo consiste en recorrer todas las aristas haciendo unión de sus extremos. Si los extremos de una arista ya pertenecen a la misma clase de equivalencia el grafo contiene un ciclo. Se realizan A operaciones de unión sobre V elementos, por lo que utilizando unión rápida por tamaño y compresión de caminos el coste de todas las operaciones está en $O(V + A \lg^* V)$.

La respuesta correcta es: $O(V + A \lg^* V)$

Pregunta 25

Sin contestar

Se puntúa como 0 sobre 1,00

En el algoritmo de Floyd con grafos sin ciclos de coste negativo los elementos de la diagonal principal siempre valen 0.

Seleccione una:

- ☐ a. Verdadero
- ☐ b. Falso

Verdadero. Los elementos de la diagonal principal en el algoritmo de Floyd representan el coste del camino mínimo de un vértice a sí mismo, que es 0 si no existen ciclos de coste negativo. En caso de existir ciclos de coste negativo, el coste de los caminos mínimos de los vértices pertenecientes a dicho ciclo a sí mismos no está acotado inferiormente.

La respuesta correcta es: Verdadero